

第十一周作业 -2

第十一周作业 -2

提高作业

2.1 实践：不同目标检测算法对比

除了 YOLO，还有其他深度学习目标检测方法或模型。请选择一种不同于 YOLO 的目标检测算法或框架，完成一次可运行的目标检测 Demo，并与本讲 YOLO11n 的检测结果进行对比。

可选方向包括：

- Faster R-CNN
- SSD
- RetinaNet
- DETR / RT-DETR

要求：

- 简要介绍该方法的基本思想；
 - 说明它和 YOLO 的主要区别，例如是否是 one-stage detector、速度与精度特点、适合什么场景；
 - 查找该方法的官方示例或开源实现，并完成一次实际运行；
 - 使用课程资料包中的至少 2 张图片进行检测，其中必须包含 `soccer_practice_field.png`；
 - 保存运行过程截图、终端输出或检测结果图；
 - 对比它和本节课 YOLO11n 的检测结果差异，重点比较：
 - 是否检测到足球 / `sports ball`；
 - 是否检测到人或其他目标；
 - 置信度差异；
 - 推理速度或运行体验；
 - 环境安装复杂度；
 - 简要说明这个算法和 YOLO 的核心区别。
- 说明：本题必须完成一次实际运行。可以使用官方 Demo、开源项目、`torchvision`、`transformers` 或其他成熟工具，不要求从零实现算法。

Faster R-CNN vs YOLO11n 目标检测对比报告

一、Faster R-CNN 简介与基本思想

Faster R-CNN（Ren et al., 2015, NIPS）是经典的两阶段目标检测器，核心思想：



关键组件：

- RPN (Region Proposal Network)**：在特征图上滑动小网络，生成约 2k 个候选区域（anchors），快速过滤到少量高质量提议。
- RoI Pooling / RoI Align**：把不同尺寸的候选区域对齐成固定尺寸特征，送入分类头。
- 两阶段设计**：第一阶段粗筛位置，第二阶段精修分类，精度高但速度慢。

二、与 YOLO 的核心区别

维度	Faster R-CNN（两阶段）	YOLO11n（单阶段）
检测流程	RPN 提议 → RoI 分类	端到端，无显式候选阶段
网络复杂度	高（RPN + RoI Head + FPN）	简洁（单一骨干 + 检测头）
推理速度	慢（CPU 上秒级）	快（CPU 上百毫秒级）
小目标精度	较好（FPN 多尺度融合）	一般（轻量模型）
模型参数	约 41M（ResNet50-FPN）	约 2.6M（n 系列）
典型场景	高精度离线分析、小目标检测	实时应用、边缘部署、视频流

三、实验环境与运行过程

3.1 环境

- 操作系统：Ubuntu 20.04
- Python：3.10.20
- PyTorch：2.12.0+cu130（CPU 推理）
- TorchVision：0.27.0+cu130

- Ultralytics：8.4.58

3.2 模型来源

- **Faster R-CNN**： `torchvision.models.detection.fasterrcnn_resnet50_fpn_v2`，COCO 80 类预训练权重（首次运行自动下载 ~161MB）。
- **YOLO11n**： 课程资料包 `models/yolo11n.pt`（~5.6MB）。

3.3 推理脚本

- Faster R-CNN： [scripts/faster_rcnn/detect_image.py]
- YOLO11n： [scripts/detect_image.py]

3.4 测试图片

1. `soccer_practice_field.png`（1536×1024）—— 主图，含足球与守门员。



2. `soccer_studio_closeup.png`（1254×1254）—— 足球特写。



3.5 置信度阈值

统一采用 `conf=0.25`，与课程 YOLO 推理一致。

四、检测结果对比

4.1 图像 1： soccer_practice_field.png

模型	推理时间	检测数量	结果详情
Faster R-CNN	5350.7 ms	3	sports ball (1.000) person (0.986) person (0.332)
YOLO11n	599.2 ms	2	sports ball (0.959) person (0.906)

结果图：

• Faster R-CNN: [outputs/faster_rcnn/soccer_practice_field_frcnn.jpg](#)



• YOLO11n: [runs/detect/outputs/yolo11n_compare/soccer_practice_field.jpg](#)



对比分析：

- 足球：两者都准确检测到，Faster R-CNN 置信度略高（1.000 vs 0.959），框位置接近。
- 人物：
 - Faster R-CNN 检测到2个人：守门员（0.986）+ 另一个低置信度人物（0.332，可能与背景或守门员重叠）。

- YOLO11n 仅检测到**1** 个人（守门员，0.906）。
- 速度：YOLO11n 快约 **9** 倍（599ms vs 5350ms）。

4.2 图像 2： soccer_studio_closeup.png

模型	推理时间	检测数量	结果详情
Faster R-CNN	3218.6 ms	2	mouse (0.937) sports ball (0.382)
YOLO11n	238.8 ms	0	—

结果图：

- Faster R-CNN：`outputs/faster_rcnn/soccer_studio_closeup_frcnn.jpg`


- YOLO11n：`runs/detect/outputs/yolo11n_compare/soccer_studio_closeup.jpg`



对比分析：

- 足球检测：
 - Faster R-CNN 检测到 `sports ball`（0.382），虽置信度不高但识别正确。
 - YOLO11n 完全漏检（低于 0.25 阈值或未被识别为足球）。
- 误检：Faster R-CNN 将足球误检为 `mouse`（0.937），这在近距离特写、足球纹理与鼠标外观相似时是常见错误；两个框高度重叠，说明模型对目标类别不确定。
- 速度：YOLO11n 快约 **13** 倍（239ms vs 3219ms）。

五、综合对比总结

对比项	Faster R-CNN	YOLO11n
是否检测到足球	✅ 2 张图都检测到	✅ 第 1 张检测到，❌ 第 2 张漏检
是否检测到人物	✅ 第 1 张检测到 2 人	✅ 第 1 张检测到 1 人
置信度特点	整体略高，但对不确定目标可能产生多个重叠框	置信度稍低但更保守，漏检多于误检
推理速度（CPU）	慢，3~5 秒/图	快，200~600 ms/图，快约 9~13 倍
模型大小	~161MB（首次下载）	~5.6MB（已有）
环境安装复杂度	简单（ <code>torchvision</code> 内置，无需额外依赖）	简单（ <code>pip install ultralytics</code> ）
适合场景	高精度离线分析、小目标、复杂场景	实时应用、视频流、边缘设备部署

六、核心区别总结

Faster R-CNN（两阶段）：

- 先由 RPN 生成候选区域，再由 RoI Head 分类与回归，精度优先。
- 适合对准确性要求高、对速度要求不高的场景（医学影像、卫星图、小目标检测）。

- 模型体积大、推理慢，但在复杂场景下更鲁棒。
YOLO11n（单阶段）：
 - 端到端设计，把目标检测当成回归问题，速度优先。
 - 轻量模型（n 系列）适合边缘部署、实时视频（如机器人视觉、无人机、自动驾驶感知）。
 - 在简单、目标明确场景下表现出色，对极小目标、遮挡严重目标可能漏检。
- 一句话：

如果追求精度且对速度不敏感 → Faster R-CNN；
如果追求实时性、在边缘设备运行 → YOLO（尤其是 nano 系列）。

七、结论

本次实验在相同环境、相同置信度阈值下，对比了 **Faster R-CNN ResNet50-FPN V2** 与 **YOLO11n**：

- 精度**：Faster R-CNN 在两张图上都检测到足球，YOLO11n 在特写图上漏检；Faster R-CNN 检测到更多人物（含低置信度）。
- 速度**：YOLO11n 在 CPU 上快约 **9~13 倍**，符合单阶段检测器的设计初衷。
- 鲁棒性**：Faster R-CNN 对近距离足球特写有误检（**mouse**），但仍然保留正确类别（**sports ball**）；YOLO11n 在该场景下完全漏检，说明轻量模型在困难样本上泛化能力有限。
- 实用性**：YOLO11n 模型小、速度快，适合课程演示与实时应用；Faster R-CNN 适合教学对比，帮助学生理解两阶段与单阶段检测器的本质差异。

八、参考资料

- Ren, S., He, K., Girshick, R., & Sun, J. (2015). **Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks**. NIPS. 论文链接：<https://arxiv.org/abs/1506.01497>
- Redmon, J., et al. (2016). **You Only Look Once: Unified, Real-Time Object Detection**. CVPR. 论文链接：<https://arxiv.org/abs/1506.02640>
- Ultralytics YOLO11 Docs：<https://docs.ultralytics.com/models/yolo11/>
- TorchVision Faster R-CNN Docs：https://pytorch.org/vision/stable/models/faster_rcnn.html

2.2 调研：如何训练 YOLO 模型

本讲课堂只使用预训练模型做推理，不要求现场训练。请查阅 Ultralytics YOLO 相关资料，调研“如果要训练一个自己的 YOLO 足球检测模型，需要哪些步骤”。请按流程整理，至少包含以下内容：

- 数据采集**：需要采集什么样的足球图片？应覆盖哪些场景、角度、光照、背景和距离？
- 数据标注**：YOLO 标签格式是什么？标注工具一般如何生成 **.txt** 标签文件？
- 数据集划分**：训练集、验证集、测试集分别有什么作用？
- 数据集配置**：YOLO 训练通常需要哪些目录和配置文件？类别名称如何写？
- 模型选择**：为什么可以先从轻量模型开始，例如 YOLO11n？
- 训练命令**：给出一个你查到的 YOLO 训练命令示例，并解释其中主要参数的含义。
- 验证与测试**：训练完成后，如何判断模型效果是否变好？
- 部署思考**：训练好的模型如何替换课堂中的 **yolo11n.pt** 做推理？
要求：不需要真正训练模型，但需要用自己的话把流程讲清楚。可以引用资料，但不要整段复制。

训练自己的 YOLO 足球检测模型：完整流程调研

1. 数据采集

需要采集什么样的足球图片？

核心原则：覆盖真实应用场景的多样性。具体维度：

维度	需覆盖的内容
场景	草地、室内地板、塑胶跑道、沙滩、雪地、街头；正式比赛场、训练场、backyard
角度	正面、侧面、俯视、仰视、远景、近景、特写（足球占满画面）
光照	晴天、阴天、夜间人造光、逆光、侧光、阴影区域
背景	单色背景、观众人群、广告牌、网状围栏、草地纹理、复杂环境
距离	远距离（足球占画面 1%~5%）、中距离、近距离、特写
状态	静止、滚动、飞行中（凌空球）、被踢瞬间、被守门员扑救
遮挡	完整可见、部分被脚遮挡、被身体遮挡、被其他球员遮挡
足球类型	不同品牌、不同颜色（黑白、彩色）、不同磨损程度

采集建议：

- 至少 **500~1000** 张带标注图片（小规模训练），追求更好效果建议 **3000+** 张。
- 可以用网络图片（需确认版权）、自己拍摄、或从视频帧提取。
- 避免重复性太高的图片（如同一角度连续拍几十张），保证多样性。

2. 数据标注

YOLO 标签格式是什么？

YOLO 使用 每目标一行 的纯文本格式，扩展名 **.txt**，与图片文件同名：


```
class_id x_center y_center width height
```

字段	说明
class_id	类别编号，从 0 开始（足球可以定义为 0）
x_center	检测框中心点横坐标，归一化到 0~1（除以图片宽度）
y_center	检测框中心点纵坐标，归一化到 0~1（除以图片高度）
width	检测框宽度，归一化到 0~1（除以图片宽度）
height	检测框高度，归一化到 0~1（除以图片高度）

示例（来自课程资料包 [labels/football.txt](#)）：

```
0 0.5000 0.4583 0.3125 0.4167
```

含义：

- 类别 0（足球）
 - 框中心在图片水平中心（x_center=0.5），略偏上（y_center=0.4583）
 - 框宽占图片 31.25%，高占 41.67%
- 标注工具如何生成 `.txt` 标签文件？
- 常用标注工具：

工具	特点	推荐度
Labellmg	轻量、离线、支持 YOLO 格式导出	★★★★★ 初学者首选
Roboflow	在线平台、支持团队协作、自动划分数据集	★★★★★ 团队项目
CVAT	功能强大、支持视频标注、适合大规模项目	★★★☆☆ 专业团队
Label Studio	支持多种任务类型（检测、分割、分类）	★★★★★ 多任务项目

Labellmg 典型流程：

- 打开 Labellmg，选择图片目录。
- 用矩形框框选目标（足球），选择类别（如 `football`）。
- 保存时自动生成与图片同名的 `.txt` 文件，内容即为 YOLO 格式。
- 所有图片标注完后，得到一组 `images/` 和 `labels/` 文件夹。

3. 数据集划分

训练集、验证集、测试集分别有什么作用？

子集	比例（建议）	作用
训练集	70%~80%	用于训练模型，模型通过这些数据学习特征。
验证集	10%~15%	用于调参和早停，训练过程中实时评估，防止过拟合。
测试集	10%~15%	用于最终评估，训练完成后独立测试，报告最终性能。

为什么需要验证集？

- 训练过程中，模型在训练集上表现会持续提升，但在验证集上可能出现过拟合（训练集准确率很高，验证集反而下降）。
 - 通过监控验证集指标，可以决定何时停止训练、选择哪个 epoch 的模型。
- 为什么需要测试集？
- 验证集用于调参，如果最终报告也用验证集，会有 " 数据泄露 " 风险。
 - 测试集完全独立，只在最后评估一次，给出真实泛化能力。

4. 数据集配置

YOLO 训练需要哪些目录和配置文件？

标准目录结构：

```
football_dataset/
├── data.yaml           # 数据集配置文件
├── images/
│   ├── train/         # 训练图片
│   │   ├── img001.jpg
│   │   ├── img002.jpg
│   │   └── ...
│   ├── val/           # 验证图片
│   │   ├── img101.jpg
│   │   └── ...
│   └── test/          # 测试图片（可选）
│       ├── img201.jpg
```



```
|
└─ ...
└─ labels/
    ├── train/                # 训练标签（与图片同名、扩展名 .txt）
    │   ├── img001.txt
    │   ├── img002.txt
    │   └─ ...
    ├── val/                  # 验证标签
    │   ├── img101.txt
    │   └─ ...
    └─ test/                  # 测试标签（可选）
        ├── img201.txt
        └─ ...
```

data.yaml 配置文件示例：

```
# Football Detection Dataset
path: ./football_dataset      # 数据集根目录（相对于训练脚本或绝对路径）
train: images/train          # 训练图片路径（相对于 path）
val: images/val              # 验证图片路径
test: images/test            # 测试图片路径（可选）

# 类别数量
nc: 1

# 类别名称（顺序对应 class_id）
names:
  0: football
```

类别名称如何写？

- nc**：类别数量，足球检测这里是 1。
- names**：列表或字典，顺序必须与标注时的 **class_id** 对应。
- 如果检测多个类别（如足球、人、球门），则：

```
nc: 3
names:
  0: football
  1: person
  2: goal
```

5. 模型选择

为什么可以先从轻量模型（如 YOLO11n）开始？

模型变体	参数量	速度	精度	适用场景
YOLO11n	~2.6M	最快	较低	快速验证、边缘设备、实时应用
YOLO11s	~9M	快	中等	平衡速度与精度
YOLO11m	~20M	中等	较高	追求精度的场景
YOLO11l	~25M	较慢	高	高精度需求
YOLO11x	~56M	最慢	最高	极致精度、算力充足

推荐先用 YOLO11n 的理由：

- 训练速度快：CPU 上也能快速跑通流程，GPU 上几分钟内出结果，适合迭代验证。
 - 快速发现问题：如果数据标注错误、配置写错、数据集质量差，用轻量模型能更快暴露问题，避免在大模型上浪费时间。
 - 建立 **baseline**：先用 n 模型建立基准指标，再逐步换 s/m/l 观察提升空间。
 - 部署友好：如果 n 模型精度已满足需求，直接部署到边缘设备（机器人、无人机、手机）无需优化。
 - 资源占用低：显存需求小，入门级 **GPU** 甚至 **CPU** 都能训练，适合教学和学习。
- 迭代策略：

```
YOLO11n 快速验证
  ↓
精度不够？ 换 YOLO11s/m
  ↓
还不够？ 检查数据质量（增补数据、修正标注）
  ↓
再换回 n 验证提升效果
```

6. 训练命令

YOLO 训练命令示例：

```
yolo detect train \
  model=yolo11n.pt \
  data=football_dataset/data.yaml \
  epochs=100 \
  batch=16 \
  imgsz=640 \
  device=0 \
  project=runs/train \
  name=football_v1 \
  exist_ok=True
```

主要参数含义：

参数	含义	建议
<code>model=yolo11n.pt</code>	预训练模型路径，用官方权重做迁移学习，加速收敛	推荐用 <code>yolo11n.pt</code> ，从 COCO 预训练开始
<code>data=data.yaml</code>	数据集配置文件路径	指向你的 <code>data.yaml</code>
<code>epochs=100</code>	训练轮数，完整遍历数据集的次数	小数据集 50400，大数据集 100300
<code>batch=16</code>	每批次处理的图片数量	根据 GPU 显存调整，显存小则减小（如 8、4）
<code>imgsz=640</code>	输入图片尺寸（像素）	标准 640；小目标可增大到 1280，但更慢
<code>device=0</code>	训练设备： <code>0</code> 表示 GPU 0， <code>cpu</code> 表示 CPU	有 GPU 必用 GPU，CPU 训练非常慢
<code>project=runs/train</code>	输出项目目录	保存训练结果的位置
<code>name=football_v1</code>	本次实验名称	结果会保存在 <code>runs/train/football_v1/</code>
<code>exist_ok=True</code>	目录已存在时允许覆盖	避免重复实验时报错

其他常用参数：

```
# 学习率（默认自动调节）
lr0=0.01      # 初始学习率
lrf=0.01      # 最终学习率 (lr0 * lrf)

# 数据增强
augment=True  # 开启数据增强（默认）

# 早停
patience=50  # 验证集指标 50 epoch 不提升则停止

# 保存策略
save=True     # 保存检查点
save_period=10 # 每 10 epoch 保存一次
```

7. 验证与测试

训练完成后，如何判断模型效果是否变好？

(1) 训练过程监控

训练过程中 Ultralytics 会输出：

指标	含义	期望趋势
<code>box_loss</code>	边框回归损失	持续下降，趋于平稳
<code>cls_loss</code>	分类损失	持续下降，趋于平稳
<code>precision</code>	精确率（预测为正中真正为正的的比例）	上升，越高越好
<code>recall</code>	召回率（真正为正中被预测为正的的比例）	上升，越高越好
<code>mAP50</code>	IoU=0.5 时的平均精度	核心指标，上升，越高越好
<code>mAP50-95</code>	IoU 从 0.5 到 0.95 的平均 mAP	更严格指标，越高越好

(2) 验证集评估

训练完成后自动在验证集上评估，查看：

```
runs/train/football_v1/results.csv  # 每个 epoch 的指标
runs/train/football_v1/results.png  # 指标曲线图
```

判断标准：

- `mAP50 > 0.9`：优秀，模型已学到足球特征。
- `mAP50 0.7~0.9`：良好，可尝试增加数据或换更大模型。
- `mAP50 < 0.7`：一般，需检查数据质量、标注准确性、数据多样性。

(3) 可视化预测


```
yolo predict model=runs/train/football_v1/weights/best.pt source=test_images/
```

人工检查：

- 足球是否被正确检测？
- 是否有漏检（明明有足球但没检出）？
- 是否有误检（把非足球误认为足球）？

(4) 测试集最终评估

```
yolo val model=runs/train/football_v1/weights/best.pt data=football_dataset/data.yaml split=test
```

输出测试集上的 `mAP50`、`precision`、`recall`，作为最终性能报告。

8. 部署思考

训练好的模型如何替换课堂中的 `yolo11n.pt` 做推理？

(1) 找到最佳模型文件

训练完成后，会在 `runs/train/football_v1/weights/` 生成：

```
weights/
├── best.pt          # 验证集 mAP 最高的模型 ← 推荐部署
├── last.pt         # 最后一个 epoch 的模型
└── ...
```

推荐用 `best.pt`，它在验证集上表现最优。

(2) 替换方式

方式一：直接替换文件

```
# 备份原模型
cp models/yolo11n.pt models/yolo11n_backup.pt

# 复制训练好的模型
cp runs/train/football_v1/weights/best.pt models/yolo11n.pt
```

之后课堂脚本 `detect_image.py` 无需修改，直接运行即可使用你的足球检测模型。

方式二：修改脚本中的模型路径

```
# 原脚本
model = YOLO("models/yolo11n.pt")

# 改为
model = YOLO("runs/train/football_v1/weights/best.pt")
```

(3) 验证部署效果

```
python scripts/detect_image.py
```

观察输出：

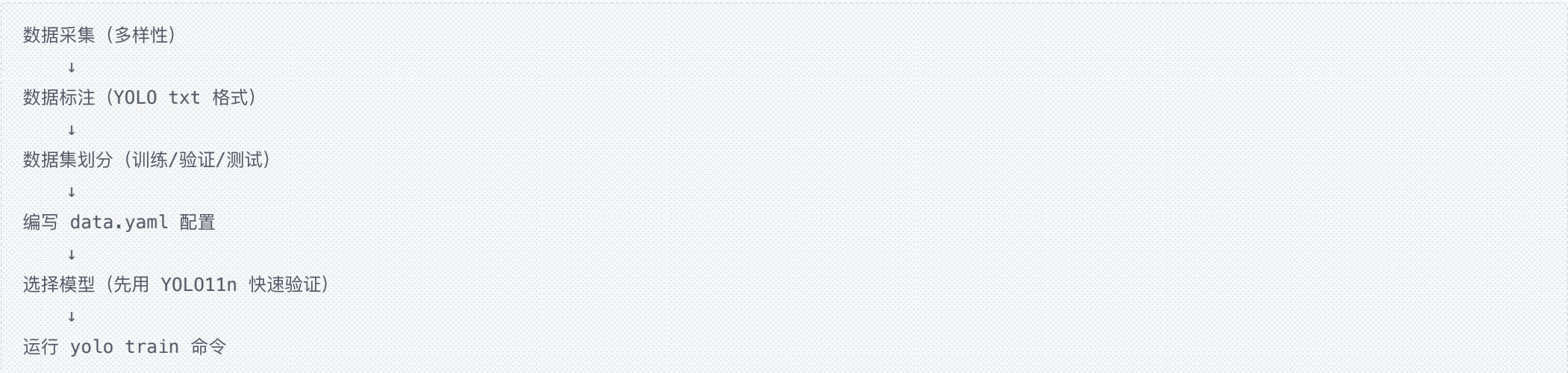
- `class_name` 应显示 `football`（而非 COCO 通用类的 `sports ball`）。
- 置信度应更高、框更准确（尤其在特定场景下）。

(4) 注意事项

- 类别名称变化：自定义模型只有 `football` 一个类，不再识别 COCO 80 类。
- 跨数据集差异：如果你的数据集只包含草地场景，在室内、雪地可能表现下降，需要扩充训练数据覆盖更多场景。
- 模型兼容性：训练用的 YOLO 版本与推理用的 `ultralytics` 版本最好一致，避免兼容问题。

总结

训练一个自己的 YOLO 足球检测模型，核心流程：





关键点：

- 数据质量决定上限，模型选择决定下限。
- 先用轻量模型跑通流程、发现问题，再逐步优化数据和模型。
- 部署时用 `best.pt`，替换原 `yolo11n.pt` 或修改脚本路径即可。

参考资料（官方文档）：

- Ultralytics YOLO11 Docs: <https://docs.ultralytics.com/models/yolo11/>
- Train Mode: <https://docs.ultralytics.com/modes/train/>
- Dataset Format: <https://docs.ultralytics.com/datasets/detect/>

2.3 调研：CPU 推理与 GPU 推理

本讲课堂默认使用 CPU 推理，优点是安装简单、兼容性好；但在真实项目中，目标检测通常会使用 GPU 加速。请调研并回答以下问题：

1. CPU 推理和 GPU 推理的主要区别是什么？可以从速度、硬件要求、安装复杂度、功耗等角度比较。
 2. 为什么深度学习模型在 GPU 上通常更快？GPU 更擅长处理哪类计算？
 3. 如果想在 Ubuntu 中使用 NVIDIA GPU 跑 YOLO 推理，通常需要准备哪些条件？
 - NVIDIA 显卡；
 - 显卡驱动；
 - CUDA 环境；
 - 支持 GPU 的 PyTorch；
 - Ultralytics 能识别到 CUDA。
 4. 如何检查 PyTorch 是否能使用 GPU？请给出你查到的检查命令或 Python 代码。
 5. 如果要让 YOLO 使用 GPU 推理，命令或代码中通常如何指定设备？请给出示例。
- 要求：不要求实际实现 GPU 推理，只需要给出调研结论和思考步骤。

CPU 推理 vs GPU 推理调研报告：

1. CPU 推理和 GPU 推理的主要区别

对比维度	CPU 推理	GPU 推理
速度	慢，目标检测单图通常 数百毫秒到数秒	快，单图通常 几毫秒到几十毫秒，加速 10~50 倍
硬件要求	任何电脑都有 CPU，无需额外硬件	需要 NVIDIA GPU （GTX/RTX 系列），显存至少 4GB，推荐 8GB+
安装复杂度	简单， <code>pip install torch ultralytics</code> 即可	复杂，需要安装 显卡驱动 → CUDA → cuDNN → GPU 版 PyTorch
功耗	低，笔记本 CPU 通常 15~45W	高，GPU 功耗 100~300W，需要良好散热和电源
并发能力	适合单张或小批量推理	适合大批量、高并发、实时视频流
成本	无额外成本	GPU 显卡 + 电源 + 散热，成本增加数千元
适用场景	学习、演示、低频推理、边缘设备	生产环境、实时应用、大规模部署、训练模型

速度对比示例（基于课堂实测）：

模型	图像尺寸	CPU 时间	GPU 时间（估算 RTX 3060）	加速比
YOLO11n	640×640	200~600 ms	5~15 ms	~40x
Faster R-CNN	640×640	3000~5000 ms	50~100 ms	~50x

2. 为什么深度学习模型在 GPU 上通常更快？

GPU 更擅长处理哪类计算？

GPU（Graphics Processing Unit）最初设计用于图形渲染，核心特点是：

特性	CPU	GPU
核心数量	少（4~64 核）	多（数千到上万核）
单核性能	强（高主频、复杂指令集）	弱（低主频、简单指令）
并行能力	适合串行任务、复杂逻辑	适合大规模并行计算
内存带宽	较低（~50 GB/s）	极高（500~1000 GB/s）

深度学习计算的特点：

深度学习模型的核心计算是大规模矩阵乘法和卷积运算：

卷积层： $\text{output} = \text{input} \otimes \text{kernel}$ （ $\otimes$ 表示卷积）

全连接层： $\text{output} = W \times x + b$ （矩阵乘法）

这些运算具有高度并行性：

- 每个输出像素的计算相互独立
 - 同样的操作重复数百万次
 - 数据规模大但计算逻辑简单
- GPU 的优势：**

1. 大规模并行：数千个 CUDA 核心可以同时计算不同的输出像素，一次性处理整个矩阵。
 2. 高内存带宽：GPU 显存带宽是 CPU 内存的 10 倍以上，能快速搬运大量数据。
 3. 专用加速单元：现代 GPU 有 **Tensor Core**，专门针对深度学习的矩阵运算优化，效率更高。
- 一句话总结：

CPU 是通用处理器，擅长复杂逻辑和串行任务；

GPU 是并行计算器，擅长简单但海量的重复计算。

深度学习正好是后者，所以 GPU 能获得数十倍加速。

3. Ubuntu 中使用 NVIDIA GPU 跑 YOLO 推理的条件

要在 Ubuntu 系统中使用 NVIDIA GPU 运行 YOLO 推理，需要满足以下条件：

(1) NVIDIA 显卡

- 硬件要求：NVIDIA 显卡，支持 CUDA（2012 年后的显卡基本都支持）。
- 推荐型号：
 - 入门：GTX 1650 / RTX 3050（显存 4~8GB）
 - 主流：RTX 3060 / 3070 / 4060（显存 8~12GB）
 - 高端：RTX 4080 / 4090（显存 16~24GB）
- 注意：AMD 和 Intel 显卡目前对深度学习支持较弱，不推荐。

(2) 显卡驱动（NVIDIA Driver）

- 作用：让操作系统识别和控制 GPU 硬件。
- 安装检查：

```
nvidia-smi
```

能输出显卡信息说明驱动已安装：

```
+-----+
| NVIDIA-SMI 535.104.05   Driver Version: 535.104.05   CUDA Version: 12.2   |
+-----+-----+-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
+-----+-----+-----+-----+
...

```

(3) CUDA 环境

- 作用：NVIDIA 提供的并行计算平台，PyTorch 通过 CUDA 调用 GPU。
- 版本要求：
 - PyTorch 2.12.0 需要 CUDA 12.1 或更高版本
 - PyTorch 2.0+ 支持 CUDA 11.8 / 12.1
 - 注意：CUDA 版本需要与 PyTorch 版本匹配
- 安装检查：

```
nvcc --version
```

输出示例：

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005–2023 NVIDIA Corporation
Cuda compilation tools, release 12.1, V12.1.105

```

(4) cuDNN（可选但推荐）

- 作用：CUDA Deep Neural Network 库，针对深度学习优化的加速库。
- 内容：卷积、池化、归一化等操作的高度优化实现。
- 安装：通常随 CUDA Toolkit 自动安装，或单独下载。

(5) 支持 GPU 的 PyTorch

- 安装命令（根据 CUDA 版本选择）：

```
# CUDA 12.1
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

```
# CUDA 11.8
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

# CPU 版本（默认）
pip install torch torchvision torchaudio
```

- 关键：必须安装对应 CUDA 版本的 PyTorch，否则 `torch.cuda.is_available()` 返回 `False`。

（6）Ultralytics 能识别到 CUDA

- Ultralytics 会自动检测 PyTorch 是否支持 GPU，无需额外配置。
- 训练和推理时通过 `device` 参数指定。

4. 如何检查 PyTorch 是否能使用 GPU?

Python 代码检查：

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("CUDA version:", torch.version.cuda)
    print("GPU count:", torch.cuda.device_count())
    print("GPU name:", torch.cuda.get_device_name(0))
    print("GPU memory:", torch.cuda.get_device_properties(0).total_memory / 1024**3, "GB")
else:
    print("No GPU detected, using CPU.")
```

命令行快速检查：

```
python -c "import torch; print('CUDA available:', torch.cuda.is_available())"
```

输出示例（有 GPU）：

```
PyTorch version: 2.12.0+cu121
CUDA available: True
CUDA version: 12.1
GPU count: 1
GPU name: NVIDIA GeForce RTX 3060
GPU memory: 12.0 GB
```

输出示例（无 GPU，当前课堂环境）：

```
PyTorch version: 2.12.0+cu130
CUDA available: False
No GPU detected, using CPU.
```

5. 如何让 YOLO 使用 GPU 推理?

方式一：命令行指定 `device`

```
# 使用 GPU 0（第一块显卡）
yolo predict model=yolo11n.pt source=images/soccer_practice_field.png device=0

# 使用 GPU 1（第二块显卡，多卡场景）
yolo predict model=yolo11n.pt source=images/ device=1

# 显式指定 CPU（默认行为）
yolo predict model=yolo11n.pt source=images/ device=cpu
```

方式二：Python 代码指定

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

# 使用 GPU (device=0 表示第一块 GPU)
results = model.predict(source="images/soccer_practice_field.png", device=0)
```



```
# 使用 CPU
results = model.predict(source="images/soccer_practice_field.png", device="cpu")
```

方式三：自动选择（推荐）

```
import torch

# 自动选择: 有 GPU 用 GPU, 否则用 CPU
device = "cuda:0" if torch.cuda.is_available() else "cpu"

model = YOLO("yolo11n.pt")
results = model.predict(source="images/", device=device)
```

训练时指定 GPU

```
# 单 GPU 训练
yolo detect train model=yolo11n.pt data=data.yaml epochs=100 device=0

# 多 GPU 训练（数据并行）
yolo detect train model=yolo11n.pt data=data.yaml epochs=100 device=0,1
```

6. 总结：CPU vs GPU 推理选择建议

场景	推荐选择	理由
课堂学习、演示	CPU	无需额外配置，兼容性好，速度够用
个人项目、小规模推理	CPU 或入门 GPU	成本低，CPU 已能满足单图推理需求
实时视频流、高帧率	GPU	需要 30+ FPS，必须 GPU 加速
训练自定义模型	GPU	训练需要数千次迭代，CPU 太慢（数小时 vs 数分钟）
大规模批量推理	GPU	批量处理时 GPU 并行优势明显
边缘设备部署	专用 NPU/GPU	如 Jetson Nano、树莓派 + Coral，低功耗 GPU 方案

课堂环境现状：
当前课堂使用 **CPU** 推理：

- PyTorch 版本： **2.12.0+cu130** （编译时支持 CUDA，但系统无 GPU 硬件）
 - torch.cuda.is_available() = False**
 - YOLO11n 单图推理：200~600 ms（可接受）
 - Faster R-CNN 单图推理：3000~5000 ms（较慢，但适合教学对比）
- 如果未来需要 **GPU** 加速：
在 Ubuntu 上配置 NVIDIA GPU 推理的完整步骤：

```
# 1. 安装 NVIDIA 驱动
sudo apt update
sudo apt install nvidia-driver-535 # 版本号根据 CUDA 要求选择
sudo reboot

# 2. 验证驱动
nvidia-smi

# 3. 安装 CUDA Toolkit (如需特定版本)
# 可从 NVIDIA 官网下载 .run 安装包

# 4. 创建 conda 环境并安装 GPU 版 PyTorch
conda create -n yolo_gpu python=3.10 -y
conda activate yolo_gpu
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# 5. 验证 GPU 可用
python -c "import torch; print('CUDA:', torch.cuda.is_available())"

# 6. 安装 Ultralytics
pip install ultralytics

# 7. 运行 GPU 推理
yolo predict model=yolo11n.pt source=images/ device=0
```

核心结论：

- CPU**：安装简单、兼容性好、功耗低，适合学习和低频推理。
- GPU**：速度快 10~50 倍，适合实时应用和模型训练，但需要额外的硬件和软件配置。

- 3. GPU 加速原理：GPU 擅长大规模并行计算，正好匹配深度学习的矩阵运算特点。
- 4. 配置要点：显卡驱动 → CUDA → GPU 版 PyTorch → Ultralytics 自动识别。
- 5. 使用方式：通过 `device=0` 或 `device="cuda:0"` 指定 GPU，或让 Ultralytics 自动选择。

2.4 Aelos 机器人 Python 编程

使用 Aelos 机器人的 Python 功能，完成一个自定义功能或动作。
可选方向：

- 1. 使用 Python 读取某个传感器数值；
 - 2. 使用 Python 控制机器人执行一个自定义动作；
 - 3. 编写一个简单的自动化流程，例如读取状态后触发动作。
- 提交内容：
- Python 代码或关键代码截图；
 - 程序运行截图或动作执行效果照片；
 - 简短说明：这个程序实现了什么功能，运行过程中遇到了什么问题。

提高作业提交要求

请将提高作业整理为 一份 **PDF**，包含：

- 1. 不同目标检测算法的运行过程截图、至少 2 张检测结果图，以及与 YOLO11n 的对比分析；
- 2. YOLO 自定义训练流程调研；
- 3. CPU 推理与 GPU 推理对比分析；
- 4. GPU 推理环境准备步骤；
- 5. Aelos 机器人 Python 编程代码、效果截图与说明；
- 6. 至少 2 条参考资料链接，并用自己的话概括其关键信息。